

AEGISPAY

Architecture V2 — Trust & Growth Layer for Agentic Commerce

AI can reason. AegisPay controls whether it is allowed to act. Razorpay only executes approved actions.

1. What is AegisPay?

AegisPay sits between AI agents (a shopping assistant, a growth agent) and payment providers (Razorpay). It does three jobs:

Job	What it means
GROW	Use AI to help a merchant sell more: upsells, cross-sells, bundles, campaigns.
SELL	Let an AI buyer find a product, build a cart and pay end-to-end.
PROTECT	Make every AI money step explainable, limited, approved, recorded and safe to recover from failure.

PROTECT is built into the architecture, not bolted on. That is why the AI can be useful and still never be trusted with money.

2. High-level architecture

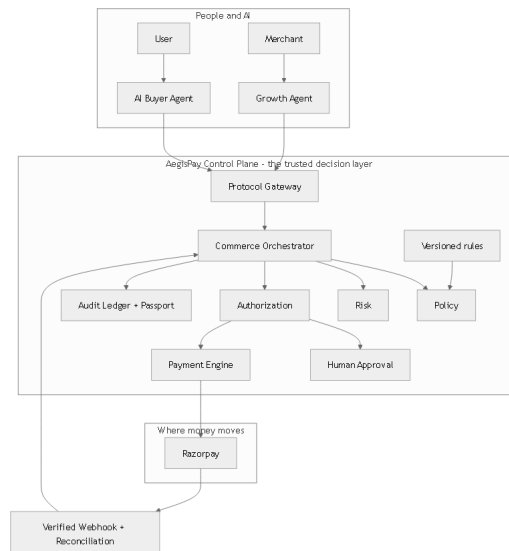


Figure 1 — People and AI (top), AegisPay control plane (middle), Razorpay (bottom)

Read it left to right: a user asks an AI buyer; everything flows into the AegisPay control plane, which checks policy, risk and authorization; only then does it call Razorpay. Razorpay replies through a verified webhook and reconciliation.

3. Component architecture

Component	What it does	Fails safely by
Commerce Orchestrator	Runs the purchase journey (intent, cart, order).	Only moves on allowed state transitions.
Policy Engine	Checks fixed limits (amount, category, hours).	Denies if it is unsure.
Risk Engine	Gives an explainable risk score + why.	Escalates to a human on doubt.
Authorization Engine	Binds a valid consent to this one transaction.	Expires and can't be reused.
Human Approval	A person decides high-value actions.	Scoped, expiring, single-use.
Payment Engine	Talks to Razorpay through one adapter.	Returns UNKNOWN rather than guessing.
Webhook Gateway	Checks and dedupes Razorpay events.	Ignores duplicates and bad signatures.
Reconciliation	Resolves UNKNOWN payments from provider truth.	Never retries blindly.
Audit + Passport	Records every decision.	Tamper-evident chain.

4. The AI / control-plane trust boundary

The most important line in the system. The AI **thinks** on one side; AegisPay **decides and acts** on the other. They only share a structured intent.



Figure 2 — The AI side hands a validated intent to the deterministic side

• The AI never sees payment keys, database credentials or unrestricted refund/payment tools.

• It only sends a typed, validated intent. The control plane re-checks everything.

• Even a manipulated AI cannot get a payment through — there is no path to money.

5. AI buyer flow (SELL)

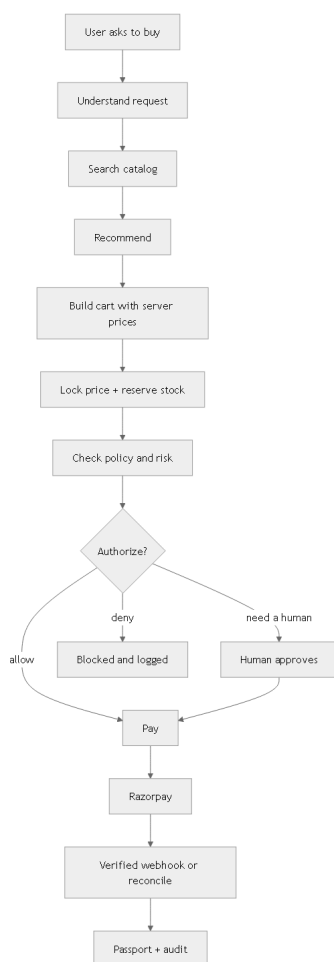


Figure 3 — A purchase from request to Passport

The AI understands, searches, recommends and builds a cart. AegisPay locks prices, checks policy/risk and authorizes. If it is high value, a human approves first. Only then does Razorpay move money.

6. Merchant growth flow (GROW)

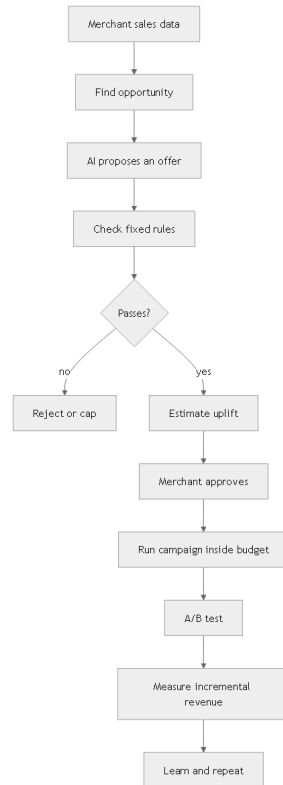


Figure 4 — From sales data to a measured, budget-limited campaign

The AI spots real patterns and proposes an offer. Fixed rules cap discount, budget and margin. A merchant approves, it runs inside budget and is A/B tested, so the uplift is measured, not claimed.

7. Policy / Risk / Authorization

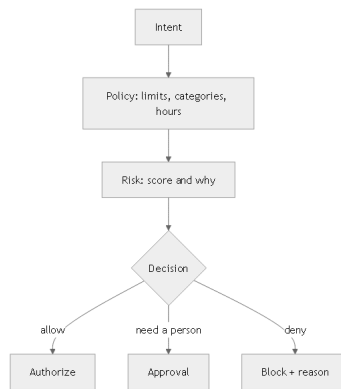


Figure 5 — Every money action passes a deterministic gate

• **Policy** is versioned and fixed by the merchant — the AI cannot change it.

• **Risk** is explainable: a score plus the reasons (new merchant, unusual amount, new device).

• **Authorization** is bound to one transaction, expires, and can't be reused.

8. Human approval

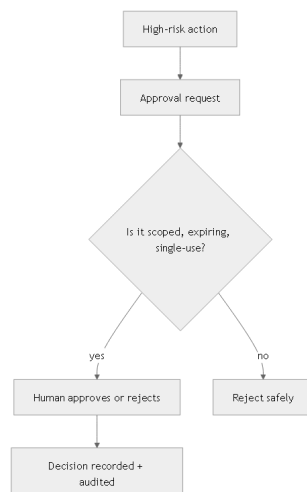


Figure 6 — A high-risk action goes to a person, safely

An approval is scoped to this exact action, expires soon, and can be used only once. It cannot be replayed or stretched into a bigger purchase.

9. Payment state machine

A payment is not a single 'success/fail' moment. It has a real lifecycle. The important state is **UNKNOWN** — when we don't know what happened yet.

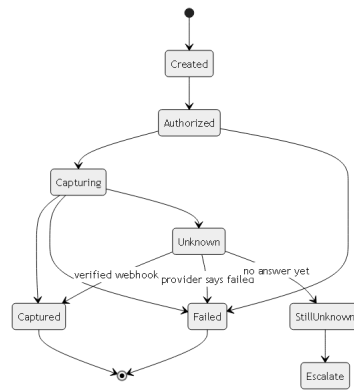


Figure 7 — Payment states, including the UNKNOWN recovery path

Front-end state is never trusted. The payment moves only on provider truth from a verified webhook or reconciliation.

10. Razorpay integration

One **PaymentProvider** interface; Razorpay is just one adapter.

Test Mode first: create order, initiate, capture, refund, fetch, verify webhook.

Provider secrets are read only inside the adapter, never in logs or the AI.

11. Webhook + idempotency

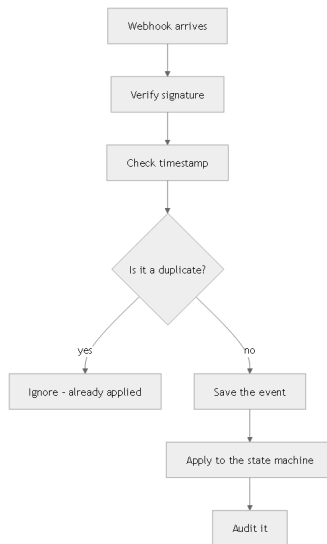


Figure 8 — Webhooks are untrusted until verified and deduped

Signature + timestamp checked first; bad ones are ignored and alerted.

A duplicate event is a no-op (no double effect).

Every money request carries an idempotency key — designed to prevent duplicate charges.

12. Reconciliation (unknown payments)

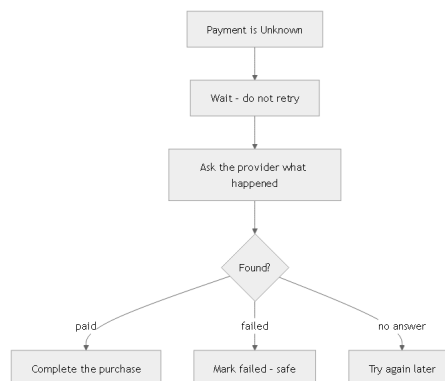


Figure 9 — Unknown payments are checked, never blindly retried

If Razorpay times out, we do not retry. We wait, ask the provider what really happened, and then finish or fail safely. This is designed so a payment cannot be charged twice.

13. Cart, price and inventory protection

Guard	What it does
Server-owned prices	Prices come from the database, never from the AI or the client.
Cart hash	Signed snapshot of the cart; a change invalidates the authorization.
Price version	A price change is detected; the checkout is re-validated.

Inventory reservation	Stock is held/locked during checkout so it can't oversell.
Cart expiry	An old cart expires; you can't pay with a stale quote.

14. Agent security

Threat	Protection
Prompt injection	Catalog is data, never instructions; intent is schema-validated.
Tool abuse	Only safe tools are exposed to the AI; no money tool.
Agent impersonation	Signed credentials, session binding, revocation.
Agent self-escalation	No tool lets an agent change its own limits or policy.

15. Multi-tenancy

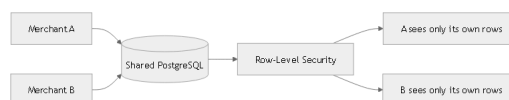


Figure 10 — Many merchants, one database, hard isolation

Every row has a tenant id, and the database enforces it with Row-Level Security. Merchant A can never read Merchant B's data, even by mistake.

16. Audit + Transaction Passport



Figure 11 — Events are linked by hash into a tamper-evident chain

Every decision is written to an append-only, hash-chained ledger. A Transaction Passport bundles the who/what/why/approval/provider so anyone can verify a purchase end-to-end. The chain is **tamper-evident**: changing an old record breaks the links.

17. AWS deployment

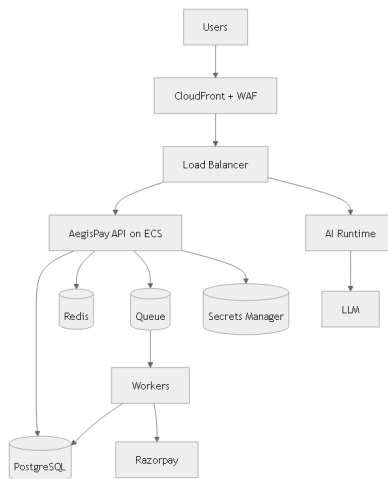


Figure 12 — Simple, boring, reliable infrastructure

Layer	Choice	Why
Compute	AWS ECS (not Kubernetes)	Simple to run, easy to scale, less to go wrong at this scale.
Data	RDS PostgreSQL multi-AZ	Auditable, correct, per-tenant security, easy backup.
Cache / locks	ElastiCache Redis	Fast caching, rate limits, locks — never the source of truth.
Queue	SQS	Reliable background work (webhooks, reconciliation).
Secrets	AWS Secrets Manager + KMS	Keys never in code, logs, or the AI.
Edge	CloudFront + WAF	TLS + protection.
AI runtime	Separate FastAPI service	No DB, no secrets, no money tools.

18. Observability + disaster recovery

Metrics: payment success/failure/unknown rate, policy/risk/approval rates, webhook duplicates, reconciliation success.

Logs: correlation, tenant, transaction and agent ids; secrets and PII are never logged.

Tracing: user → agent → tool → policy → risk → authorization → payment → webhook.

DR targets: backups + point-in-time restore, RPO ≤ 15 minutes, RTO ≤ 1 hour, multi-AZ, quarterly restore drills.

A global kill switch can stop all new AI-initiated money movement instantly if something looks wrong.

19. Production-readiness checklist

Area	Must be in place
Payments	Idempotency, state machine, webhook verification, reconciliation, refund caps.
AI	Structured output, tool allowlist, policy + risk, human approval, red-team passing.
Security	Auth (user, agent, service), secrets, encryption, rate limits, tenant isolation.
Trust	Versioned policy, explainable risk, Transaction Passport, tamper-evident audit.
Reliability	Timeouts, retries, circuit breakers, queue + DLQ, backups + DR.
Observability	Metrics, logs, tracing, alerts, dashboards.

Why AegisPay is difficult to reject

The AI can reason and recommend, but only AegisPay's deterministic control plane can authorize and execute financial actions. That is the whole product.

• **It is safe by design, not by promise.** The AI has no money keys and no payment endpoint — only a request that is always checked.

• **It handles failure honestly.** Unknown payments are reconciled rather than blindly retried; duplicates are designed out, not hoped away.

• **You can prove it.** Every decision is written to a tamper-evident audit ledger and a Transaction Passport.

• **It is simple to run.** One FastAPI service, PostgreSQL + Redis + one queue on AWS ECS — no unnecessary complexity.

• **It grows and sells.** GROW lifts revenue with bounded risk; SELL lets AI buyers transact end-to-end. PROTECT keeps both safe.